Figma for people who think in code

Week 1, Lecture 1 · Design for Builders: Ship Beautiful Products as a Founder

Autumn 2026

What we will cover

- Why frames are not groups (and why it matters)
- Auto-layout as a direct mapping of CSS flexbox
- Components, variants, and component properties
- Styles vs variables: static values vs dynamic systems

Part 1: frames vs groups

The frame is the unit of layout

- A group is a selection shortcut: it has no layout behavior of its own
- A frame is a layout container: it has width, height, padding, clipping, and auto-layout
- Every screen in Figma is a top-level frame (previously called an artboard)
- Nested frames are how you compose complex layouts

Think of a group as a `<span>` and a frame as a `<div>`.

What a frame can do that a group cannot

- Clip content to its bounds (overflow: hidden)
- Set background fill and corner radius as a contained surface
- Use auto-layout (the equivalent of `display: flex`)
- Become a component or a component instance
- Constrain children when resized (position: relative with anchored children)

When to use a group

- Temporarily selecting multiple layers you want to move together
- Grouping decorative elements that do not need layout rules (illustrations, icons you have not yet componentized)
- Any time you want the container to hug its children without setting explicit dimensions

If you find yourself reaching for a group to solve a layout problem, you probably want a frame.

Part 2: auto-layout as flexbox

Auto-layout is CSS flexbox, renamed

Figma term	CSS equivalent
Direction: Horizontal	`flex-direction: row`
Direction: Vertical	`flex-direction: column`
Gap	`gap`
Horizontal padding	`padding-left / padding-right`
Vertical padding	`padding-top / padding-bottom`
Align: Center	`align-items: center`

Resizing behaviors: the four modes

- **Fixed** — explicit pixel width/height (`width: 320px`)
- **Hug** — shrink to fit children (`width: fit-content`)
- **Fill** — expand to fill the parent container (`flex: 1`)
- **Min/Max** — constrained fill (`min-width / max-width` with `flex: 1`)

Mix these on parent and child to build responsive layouts without absolute positioning.

A card in practice

```
Card frame (Vertical, 16px gap, 24px padding, Hug height, 320px Fixed width)
  ├── Image frame (Fill width, 160px Fixed height)
  ├── Title text (Hug)
  ├── Body text (Fill width, Hug height)
  └── Button (Hug width, Hug height)
```

Change the body text length and the card grows vertically. Change the card width and the image and text fill it. This is exactly how a flex column works in CSS.

Nesting auto-layout frames

- Outer frame: `flex-direction: row`, gap between cards
- Inner frame (each card): `flex-direction: column`, padding inside
- Nesting is how you build a feature section, a pricing table, or a nav bar

The rule: each frame controls layout for its own direct children. It does not reach into grandchildren.

Part 3: components, variants, properties

What a component is

- A master definition you create once (the main component)
- Any number of instances that inherit changes from the main
- Change the main component: all instances update
- Override an instance: that override survives future main-component changes (unless the property is removed)

This is the React component model. The main component is the function definition. Each instance is a call to that function.

Variants: the prop system

- A variant set is a collection of related components grouped under one name
- Each variant is one combination of property values (size, state, color)
- Switching variants in an instance = passing a different prop value
- Figma auto-generates a grid showing every variant combination

Example: Button with `size` (sm, md, lg) and `variant` (primary, secondary, ghost) = 9 variants.

Component properties

Three kinds you will use constantly:

1. **Boolean property** — show or hide a child layer (e.g. show/hide an icon)
2. **Text property** — override a text layer from the instance without entering edit mode
3. **Instance swap property** — swap a nested component for a different one (e.g. swap an icon)

Component properties let you expose just the right controls to users of your component, without exposing the full layer structure.

Where the abstraction breaks

- Figma components have no conditional rendering logic: you must use boolean properties + hidden layers to simulate `if` statements
- There is no `children` prop: you cannot pass arbitrary content into a component slot (prototyping tools like this are in beta)
- Component state (hover, focus, active) requires separate variants plus interactive connections, not CSS pseudo-classes
- A deeply nested component hierarchy becomes slow to render in large files

(Figma, 2023; Frost, 2016)

Part 4: styles vs variables

Styles: static named values

- A style gives a name to a set of properties: fill color, text style, effect
- Applying a style to a layer links that layer to the named value
- Change the style definition: all layers using it update
- Styles have been in Figma since 2018 and work on the free plan

Variables: dynamic, mode-aware values

- A variable stores a single value (color, number, string, boolean) in a named collection
- A collection can have multiple modes (Light / Dark, Desktop / Mobile, English / French)
- Switch the mode on a frame: every variable in that frame resolves to its mode value
- This is how one-click light/dark mode works in Figma

Figma (2023): “Variables let you create designs that adapt to different contexts without duplicating layers.”

When to use each

Use styles for…	Use variables for…
Text styles (type scale)	Color tokens that need dark mode
Effect styles (shadows)	Spacing values used in components
Color palettes you share with legacy libraries	Semantic roles (color-surface-base, color-text-primary)

For a starter kit in Week 1: use color styles for your palette. Migrate to variables when you add dark mode (Week 4).

> Components provide a way to split the UI into independent, reusable pieces, and to think about each piece in isolation.

Take-aways

1. Frames have layout behavior; groups do not. Default to frames for anything that needs auto-layout, clipping, or resizing rules.
2. Auto-layout maps directly to CSS flexbox. If you know flexbox, you already know how to reason about auto-layout.
3. Components + variants = a prop system for design. Boolean, text, and instance swap properties are the three properties you need most.
4. Styles are static. Variables are dynamic and mode-aware. Use styles for your type scale; use variables for anything that needs to change across contexts.

What’s next

- **Reading:** Week 1 reading covers frames, auto-layout, components, and the styles vs variables decision in depth
- **Lecture 2 (this week):** File structure, team libraries, and plugins
- **Section (this week):** Rebuild a landing-page section pixel-for-pixel in Figma
- **HW 1 (out this week):** Build a 12-component Figma starter kit for your product